

Understanding performance bottlenecks

OPTIMIZING CODE IN JAVA



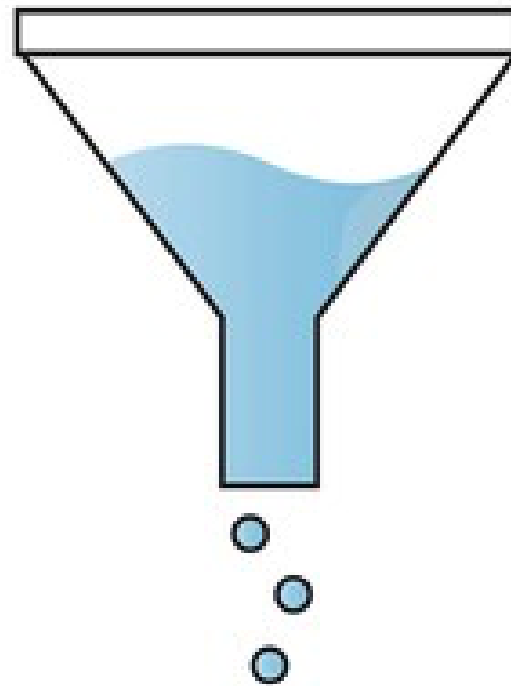
Pavlos Kosmetatos

Lead Engineer @Wealthyhood

What are performance bottlenecks?

*They are **constraints** that limit our application's speed and efficiency.*

Imagine pouring water through a funnel - the narrowest part determines the flow rate!



Types of bottlenecks

CPU-bound bottlenecks

- Complex calculations
- Inefficient algorithms

I/O-bound bottlenecks

- Database operations
- Network calls
- File operations

Memory-bound bottlenecks (covered in later lessons)

Measuring performance with System.nanoTime()

- `System.nanoTime()` - High-precision time measurement
 - Measures elapsed time
 - Returns nanoseconds (1 billion = 1 second) as a `long` number
- Unlike `System.currentTimeMillis()`, measures wall clock time

```
long startTime = System.nanoTime();  
// Code to measure  
long endTime = System.nanoTime();  
long duration = endTime - startTime;
```

Using System.nanoTime()

```
// Measuring ArrayList.contains() performance
import java.util.ArrayList;
import java.util.List;

List<Integer> numbers = new ArrayList<>();
for (int i = 0; i < 1000000; i++) {
    numbers.add(i);
}

long startTime = System.nanoTime();
boolean found = numbers.contains(999999); // O(n) operation
long endTime = System.nanoTime();
```

Using System.nanoTime()

```
long duration = endTime - startTime;  
System.out.println("Time taken: " + duration + " ns");  
// Convert to milliseconds  
System.out.println("Time taken: " + duration / 1_000_000.0 + " ms");
```

```
Time taken: 12390000000 ns  
Time taken: 1239ms
```

- `ArrayList.contains()` is an $O(n)$ operation

Best practices for performance measurement

- Run multiple measurements

Run multiple measurements

```
long totalTime = 0;
for (int i = 0; i < 3; i++) {
    long start = System.nanoTime();
    findElementLinear(data, target);
    long iterationTime = System.nanoTime() - start;
    totalTime += iterationTime;
    System.out.println("Run " + (i+1) + ": " + iterationTime + " ns");
}
double averageTime = totalTime / 3.0;
System.out.println("Average: " + (long)averageTime + " ns");
```

```
Run 1: 3245678 ns
Run 2: 3198432 ns
Run 3: 3301234 ns
Average: 3248448 ns
```


Best practices for performance measurement

- Run multiple measurements
- Compare relative performance

```
// Assume we have calculated avgTimeA and avgTimeB already  
System.out.println("Operation A is" + (avgTimeA / avgTimeB) + "x slower");
```

```
Operation A is 4x slower
```

Let's practice!

OPTIMIZING CODE IN JAVA

Understanding JVM Memory Architecture

OPTIMIZING CODE IN JAVA



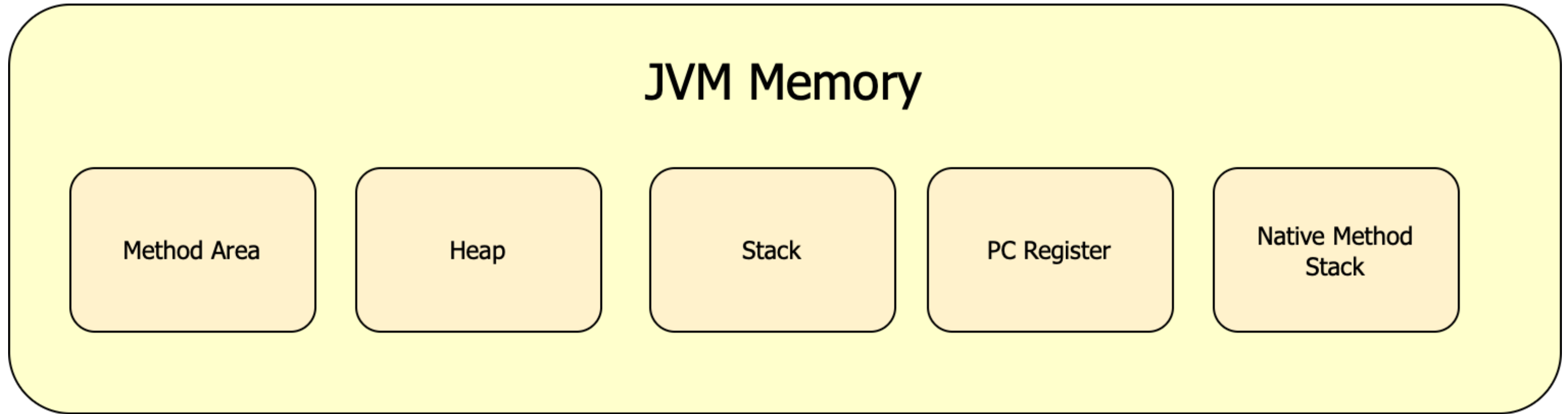
Pavlos Kosmetatos

Lead Engineer @Wealthyhood

What is the JVM?

- Platform-independent execution environment
- Converts bytecode (.class) to machine code
- Memory management and garbage collection
- Just-In-Time compilation

JVM memory architecture overview



Stack memory

- Memory area for method execution and local variables
- LIFO (Last-In-First-Out) structure
- Stores:
 - Local variables
 - Primitive data types
 - Method call information
- `StackOverflowError` if size is exceeded

Heap memory

- Stores all objects and arrays
- `new` creates an object and allocates it in the heap
- Managed by garbage collector
- Dynamic size
- Divided into generations

Memory in action

```
public void processData() {  
    int count = 0;           // Stack  
    String name = "Java";    // Heap  
    List<String> items =      // Reference on Stack  
        new ArrayList<>();    // Object on Heap  
    items.add(new String("Item")); // Object on Heap  
}
```


Memory-efficient coding patterns

```
// Inefficient
List<String> list1 = new ArrayList<>();
for (int i = 0; i < 1000; i++) {
    String s = new String("value"); // 1000 objects
    list1.add(s);
}
```

```
// Better
List<String> list2 = new ArrayList<>();
String s = "value"; // Single object
for (int i = 0; i < 1000; i++) {
    list2.add(s);
}
```

Summary

- JVM manages memory through (mainly) stack & heap areas
- Stack memory -> fast, stores primitives and references
- Heap memory -> Stores objects

Let's practice!

OPTIMIZING CODE IN JAVA

Performance metrics and measurement

OPTIMIZING CODE IN JAVA



Pavlos Kosmetatos

Lead Engineer @Wealthyhood

Beyond time measurements

- Previous videos
 - Measure time taken for operations
 - Memory management in the JVM
- Moving beyond time measurements
- Memory metrics
- CPU utilization metrics

Memory metrics with runtime

```
// Get the Runtime instance
Runtime runtime = Runtime.getRuntime();

// Memory metrics in bytes
long totalMemory = runtime.totalMemory(); // Memory currently allocated
long freeMemory = runtime.freeMemory(); // Memory that is unused
long maxMemory = runtime.maxMemory(); // Maximum memory JVM will attempt to use
```

- All return `long` numbers representing bytes

Understanding memory metrics

- `maxMemory()` : Maximum memory JVM will attempt to use (set by -Xmx)
- `totalMemory()` : Currently allocated memory to the JVM
- `freeMemory()` : Unused memory
- `usedMemory()` : `totalMemory - freeMemory`
- JVM can increase `totalMemory` up to `maxMemory` as needed

Memory metrics in practice

```
// Create a utility method for readable memory values
public static double getUsedMemoryMB() {
    Runtime runtime = Runtime.getRuntime();
    long memory = runtime.totalMemory() - runtime.freeMemory();
    return memory / (1024.0 * 1024.0); // Convert to MB
}

// After allocation
System.out.println("Memory used: " + getUsedMemoryMB() + " MB");
```

Memory used: 5.23 MB

CPU utilization measurement

```
import java.lang.management.*;

ThreadMXBean threadMXBean = ManagementFactory.getThreadMXBean();

// Check if CPU time measurement is supported
if (threadMXBean.isThreadCpuTimeSupported()) {
    // Enable CPU time measurement
    threadMXBean.setThreadCpuTimeEnabled(true);

    // Get CPU time in nanoseconds
    long cpuTimeNs = threadMXBean.getThreadCpuTime(Thread.currentThread().threadId());
}
```

Java profiling tools

- JVisualVM: Visual interface for monitoring memory, CPU and more
- jmap: Command-line memory analysis tool
- These tools go beyond the simple metrics we've discussed!

Let's practice!

OPTIMIZING CODE IN JAVA

Garbage collection & Just-In-Time (JIT) compilation

OPTIMIZING CODE IN JAVA



Pavlos Kosmetatos

Lead Engineer @Wealthyhood

Garbage collection - the basics

Manual Memory Management (C/C++)

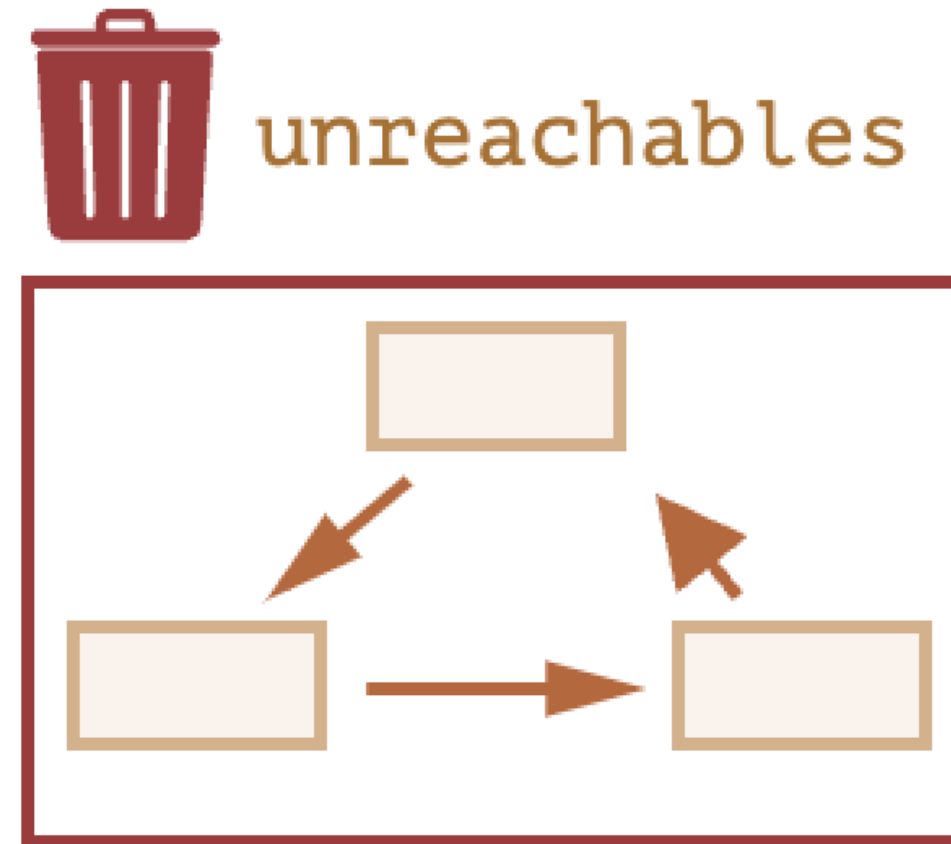
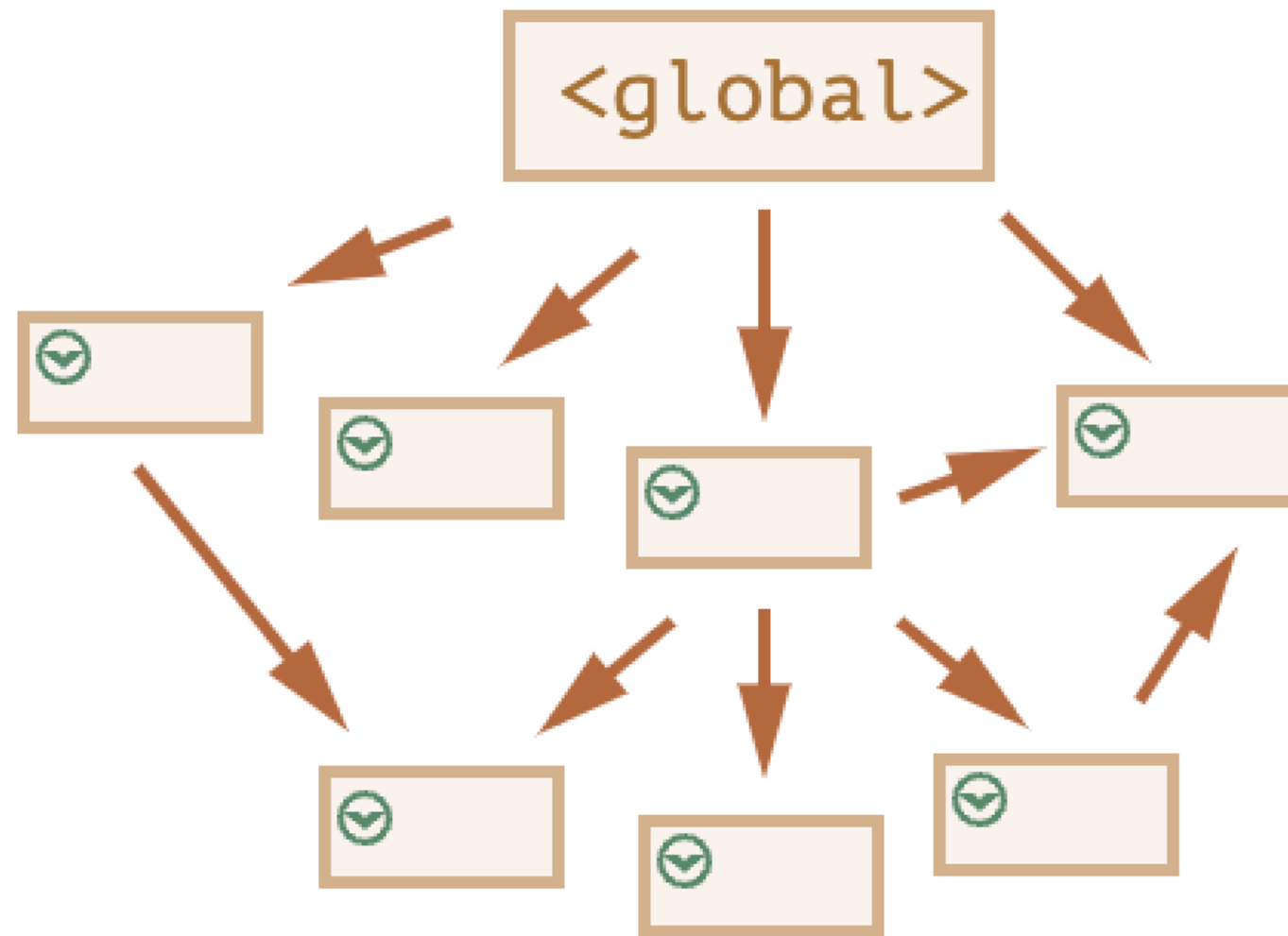
- Programmer allocates & frees memory explicitly
- Prone to leaks

Java's Garbage Collection

- Automatic memory allocation
- Automatic detection of unused objects
- Automatic memory reclamation

How garbage collection works

- Mark: Identify reachable objects from roots
- Sweep: Remove unreachable objects



Garbage collection and performance

GC Performance Trade-offs

- Simplifies development, but comes with performance considerations

GC Algorithms

- There are several garbage collection algorithms offered by modern JVMs
- The choice depends on the application's requirements
- A deep-dive guide regarding GC algorithms can be found [here](https://www.baeldung.com/jvm-garbage-collectors)

¹ <https://www.baeldung.com/jvm-garbage-collectors>

GC performance impact

```
// Bad - Creating many short-lived objects
for (int i = 0; i < 10000000; i++) {
    new Object(); // Immediately becomes garbage
}
```

```
// vs. Reusing objects
Object obj;
for (int i = 0; i < 10000000; i++) {
    obj = new Object(); // Only one reference
}
```


GC-friendly coding

```
// Avoid this in performance-critical code
String result = "";
for (int i = 0; i < 1000000; i++) {
    result += i; // Creates a new String each time
}
```

```
// Better approach
StringBuilder builder = new StringBuilder();
for (int i = 0; i < 1000000; i++) {
    builder.append(i); // Reuses the same buffer
}
String result = builder.toString();
```

Just-In-Time compilation

- JVM execution process:
 - Java source -> Bytecode
 - Bytecode -> Interpreted execution
 - Hot methods -> Native machine code
- Optimizes frequently executed code

JIT-friendly code patterns

- Write predictable code

```
// Predictable branches help JIT optimization
if (value > 0) {
    // Code A - frequently taken path
} else {
    // Code B - rarely taken path
}
```

- Create focused, smaller methods
- Small, frequently called methods are ideal candidates for inlining
- Complex code may be optimized less effectively

Summary

- Minimize object creation in performance-critical paths
- Write predictable code that aligns with JIT optimization capabilities
- Focus optimization on the code that actually matters by profiling first

Let's practice!

OPTIMIZING CODE IN JAVA